

نمط شرح مختصر

الكلاسات الثلاث وتسلسل العمل

كلاس 1.

هو الكائن الأصلي الذي نريد نحفظ حالته. ينشئ الـ من بياناته ويعيد تحميلها منه.

من الحالة الحالية ينشئ:

```
public IMemento createMemento() {  
    return new Memento(options, isSelected);  
}
```

: يستعيد الحالة من الـ

```
public void restoreState(IMemento memento) {  
    Memento m = (Memento) memento; //   
    options = m.getOptions();  
    isSelected = m.isSelected();  
}
```

فقط يعمل لـ بقية الكلاسات ترى فقط ولا تقدر تفتحه.

كلاس 2.

كائن سلمي يخزن البيانات فقط، لا يغير شيء ولا يتخذ قرارات.

```
public Memento(int[] options, boolean isSelected) {  
    this.options = options.clone(); //   
    this.isSelected = isSelected;  
    this.timeStamp = LocalDateTime.now().toString();  
}
```

بدون . سيتشارك والـ نفس المصفوفة، وستتغير البيانات المحفوظة بالخطأ.

واجهة تخفي البيانات عن الـ :

```
public interface IMemento {  
    String toString(); //   
}
```

كلاس 3.

يدير قوائم الـ لكنه لا يفتح الـ أبداً، فقط ينقله.

بالتسلسل:

```
public void undo() {
saveToRedoHistory(); // 1. ■■■■ ■■■■■■ ■■■■■■ ■■ redo
IMemento prev = undoHistory
.remove(undoHistory.size()-1); // 2. ■■ ■■■■ ■■■■ ■■■■■■
model.restoreState(prev); // 3. ■■■■■■ ■■ Model ■■■■ ■■ ■■■■■■
gui.updateGui(); // 4. ■■■■ ■■■■■■
}
```

بالتسلسل:

```
public void redo() {
saveToUndoHistory(); // 1. ■■■■ ■■■■■■ ■■■■■■ ■■ undo
IMemento next = redoHistory
.remove(redoHistory.size()-1); // 2. ■■ ■■■■ ■■■■ ■■■■■■ redo
model.restoreState(next); // 3. ■■■■■■ ■■ Model ■■■■ ■■ ■■■■■■
gui.updateGui(); // 4. ■■■■ ■■■■■■
}
```

تسلسل العمل الكامل

الخطوة	من يتصرف	ماذا يحدث
1	المستخدم	يضغط على مستطيل
2		يستدعي
3		ينشئ ويضيفه لـ
4		يحدث
5		يمسح تغيير جديد
6	المستخدم	يضغط
7		يحفظ الحالة الحالية في
8		يأخذ آخر من
9		يستعيد و
10		تحديث الواجهة

الأهم: لا يفتح الـ أبداً فقط ينقله. هو الوحيد الذي يقرأ ويكتب محتوى الـ .

القاعدة